LAB EXPERIENCE COURSE

Report No.4

Implementation of Entropy Polarization-Based Data Compression Without Frozen Set Construction

Student Name

Bauyrzhan Kurmangaliyev

Date

17.05.2026

I. Introduction.

During the previous study, the design of frozen bit sets was essential for the efficient compression of binary sources. However, is there a possibility to omit this burden and find out another way to pass most erroneous bits? The study conducted by Ren and Zhao, 2025 has proposed a novel method to compress data without explicit construction of frozen bits. This work has been roughly divided in three parts:

a) Implementation of core principles of proposed method with a standard Successive Cancellation (SC) decoder

b) Implementation of proposed encoder/decoder scheme with non-standard SC decoder based on Log-Likelihood Ratios (LLRs)

c) Extension of proposed method with fixed threshold.

II. Task 1: Algorithm implementation up to Section III.B

The main idea behind construction free strategy is that instead of relying on expected entropy , the encoder mimics the decoder behavior by calculating exact conditional entropy.

Exact conditional entropy is a binary entropy calculated with probabilities defined in Equation 3 of the paper. This refers to SC decoder output probabilities which are then fed to the Maximum Likelihood decision. Since we are working with probabilities, instead of LLRs, we implement the standard SC decoder proposed by Arikan.

Encoder Structure: At leaf node, we compute the conditional entropy (binary), store it and perform ML decoding. Then we check if the decoded bit is equal to the true bit. If it does not, store its index in the error set.

At the intermediate nodes, the encoder propagates and transforms probabilities using check and bit nodes, and recombines decision bits.

As a result, the encoder computes the threshold entropy as the minimum entropy of entropies being part of the error set. Additionally, it forms set G: as the set of erroneous bits, where indices represent the positions in the sequence whose exact conditional entropies are greater than or equal to this threshold . The output of the encoder then consists of this threshold value along with the subsequence of source bits u_G located at those specific indices.

Decoder structure: At leaf node, we calculate the entropies and perform a check, if entropy at current index is greater than threshold, we set u_G bit at current position as the decoded bit value.

Otherwise, we perform ML decisions.

In order to observe the Expected Encoding Rate, we run Monte Carlo simulations to find expected encoding length, defined as cardinality of set G.

III. Task 2: Algorithm implementation with LLR

Since, LLR based implementation of SC decoder is much widely used, we adapt the aforementioned algorithm for this case. Let's call it a non-standard SC decoder. The encoder and decoder idea remains similar, however we must modify the error set, threshold and set G formed by the threshold. These modifications are outlined with equations (9), (10) in the paper.

IV. Task 3: Harnessing a Fixed Threshold (LLR based)

Previously determined fixed threshold is highly dependent on data and statistical realization. This results in storage overhead which is not desirable for practical applications. To leverage this, we introduce a fixed threshold, defined in equation 15 within the LLR framework.

Encoder: Similar to Section III, however, we must return three main ingredients:

1. Fixed threshold
2. G^*_{fix} The list of global indices where the ML estimator failed, but their LLR magnitudes were too high.
3. $u_{G,fixed}$ Source bits that are inside the high-entropy set, ($|LLR_i| \leq LLR_{threshold,fixed}$)

Decoder: If the LLR in current index is less than fixed LLR sequence, we force the decoder output as $u_{G,fixed}$ at the current index. If the current index is inside G^*_{fix}, we flip this bit after the hard decision. Otherwise, we perform regular decisions.

Finally, we run Monte Carlo simulations to find Expected Encoding Rate, by following equation 14.

V. Results

All simulations have been done for $N = 1024$, 1000 Monte Carlo iterations.

Figure 1 represents the comparison of Expected Encoding Rate of two algorithms. It can be seen that construction-free strategy brings substantial improvements in encoding rate.

Figure 2 represents the Expected Encoding Rate for standard SC encoder/decoder, it is immediately apparent that at low probability there is a sudden jump in rate. This problem has been studied and it was observed that at low probability the threshold is highly unstable. Specifically, either the Error Set is empty (threshold zero, cardinality of G is full) or it is populated and the rate results in expected values. I am currently observing this problem and I will provide a solid answer later.

Figure 2 represents the Expected Encoding Rate for LLR based implementation of this algorithm. It is seen that the performance of this algorithm matches with the previous one, being more stable at the outliers.

Figure 3 shows the result of the same algorithm for the fixed threshold. It has been observed that

its performance is slightly worse than previous modifications, however it is not dependent on data and does not suffer from memory overhead.

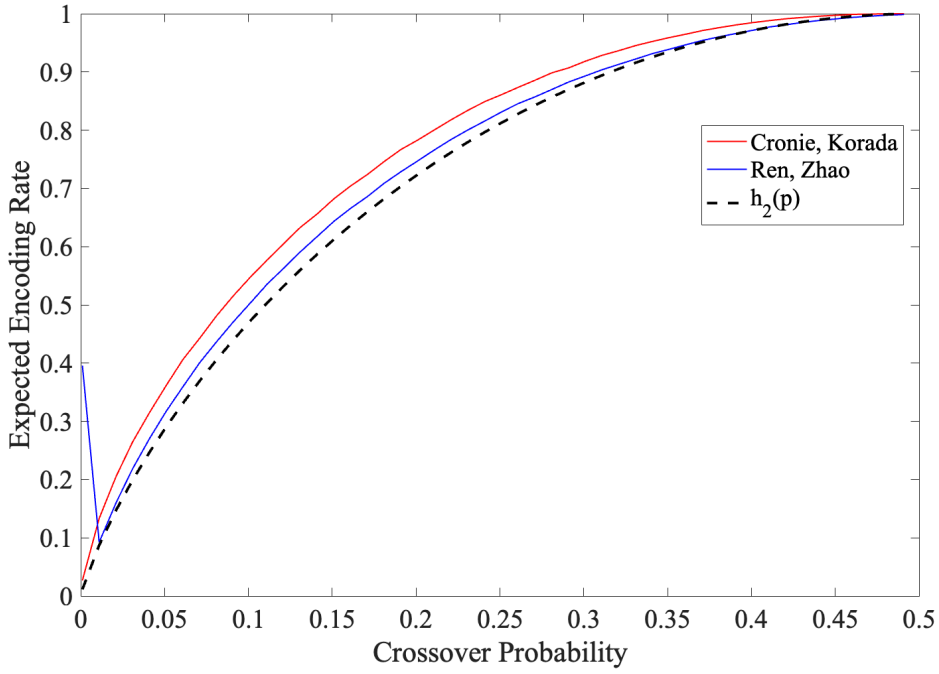

Figure 1. Comparison of two compression algorithms

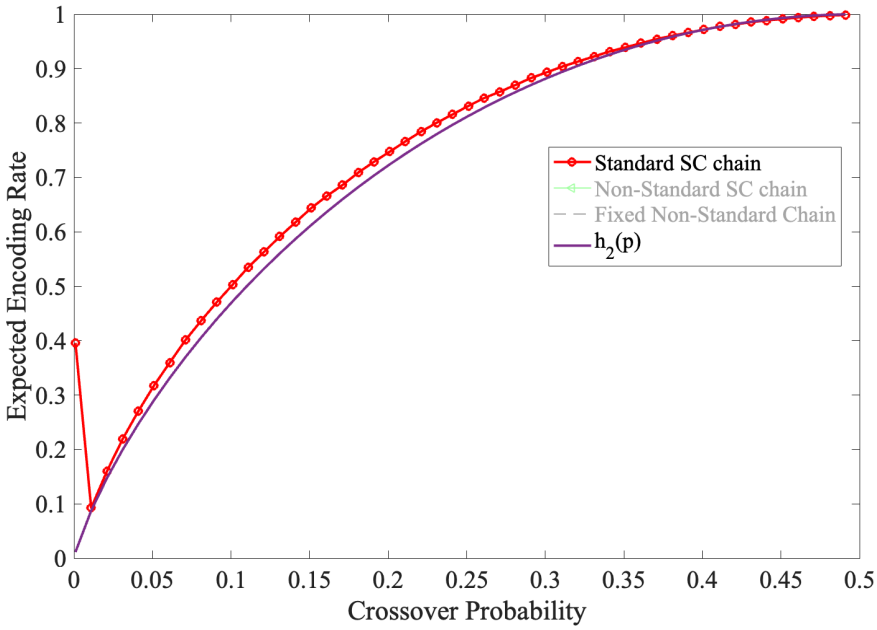

Figure 2. Rate of Standard SC chain

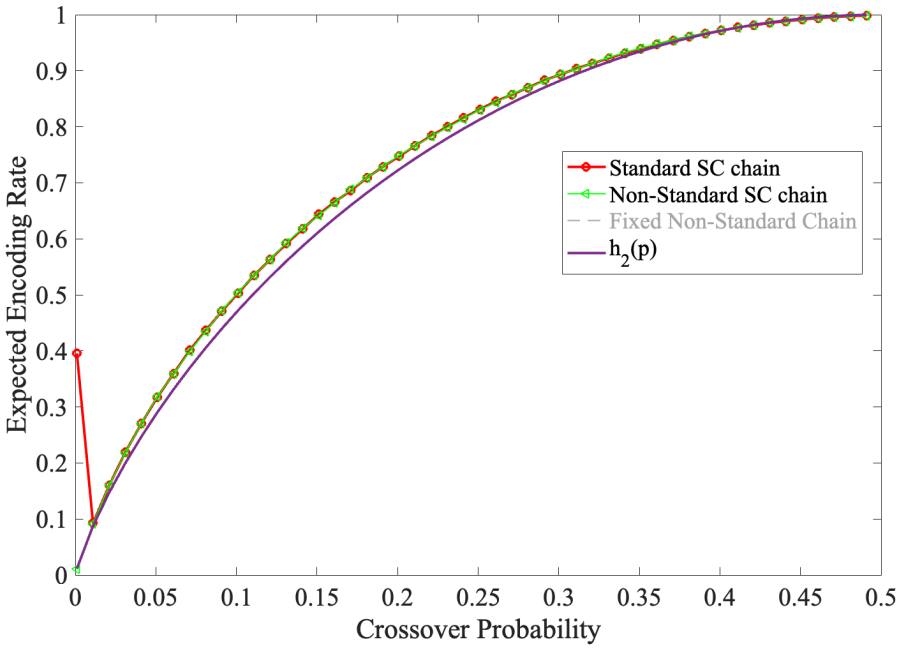

Figure 3. Standard + Non Standard SC chain

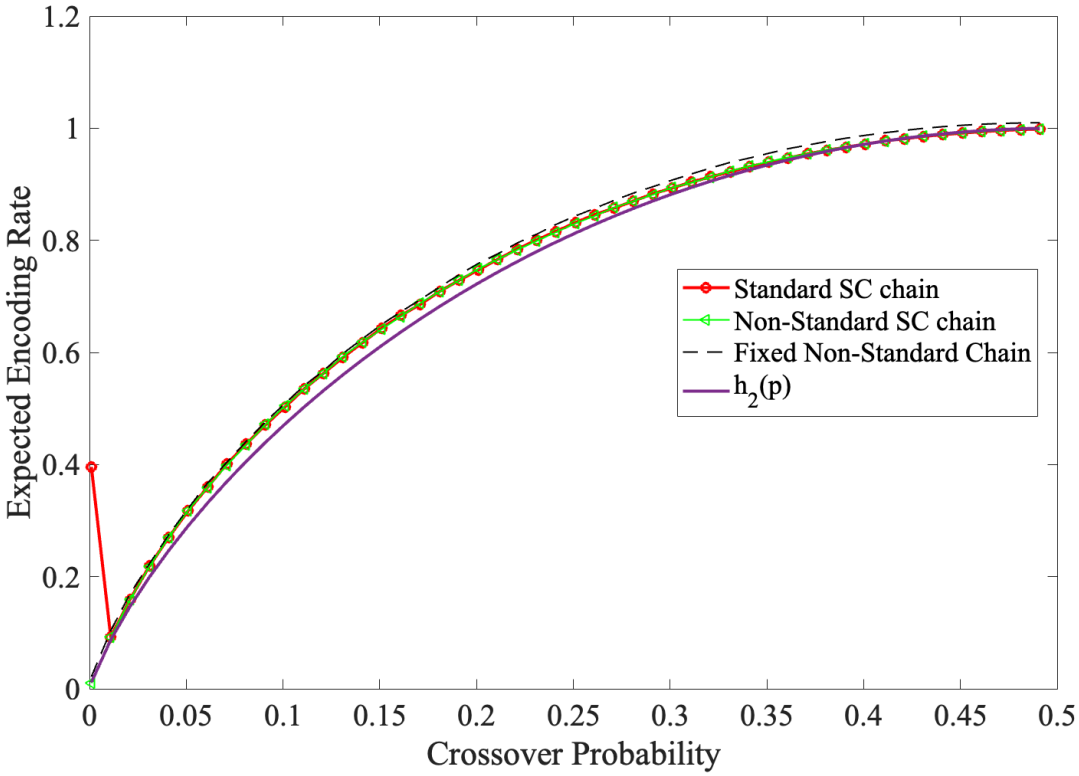

Figure 4. Full Performance Graph with Fixed Non-Standard chain

Code:
https://github.com/Uleee/Lab_Experience_Source_Coding_Polimi26